Karlsruher Institut für Technologie

Institut für Programmstrukturen und Datenorganisation

Prof. Dr.-Ing. Gregor Snelting
gregor.snelting@kit.edu

Prof. Dr. Ralf H. Reussner
reussner@kit.edu

Klausur Programmierparadigmen — Beispiellösung

SS23, 29. September 2023, 8:00 – 10:00 Uhr

Zugelassene Hilfsmittel:
Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

Bearbeitungszeit: 120 min

Aufgabe 1 (Haskell: Turtle-Grafik) [17 Punkte]

Für diese Aufgabe brauchen wir einige Hilfsfunktionen rund um zweidimensionale Vektoren. Der Typ für 2D-Vektoren sei definiert als:

```
type Vec = (Double, Double) -- (x-Koordinate, y-Koordinate)
```

Implementieren Sie die folgenden Hilfsfunktionen.
Geben Sie zu Ihren Lösungen von (a) bis (c) eine Typsignatur an.

(a) Eine Funktion `vadd`, mit der zwei Vektoren addiert werden können, z. B. [2 Punkte]
`vadd (1.0, 2.0) (3.0, 3.0)` ⇒ `(4.0, 5.0)`

(b) Eine Funktion `vscale`, mit der ein Skalar und ein Vektor multipliziert werden können, z. B. `vscale 3.0 (1.0, 2.0)` ⇒ `(3.0, 6.0)` [2 Punkte]

(c) Eine Funktion `unit`, die zu einem gegebenen Winkel (im Bogenmaß) den Einheitsvektor in diese Richtung zurückgibt. Dabei entspreche wie üblich der Winkel 0 der x-Richtung und der Winkel steige gegen den Uhrzeigersinn, z. B. [2 Punkte]
`unit (pi / 2)` ⇒ `Vec 0.0 1.0`
Hinweis: Die Sinus- und Kosinusfunktion sind in Haskell als `sin :: Double -> Double` bzw. `cos :: Double -> Double` definiert.

Mit diesen Funktionen wollen wir nun einen einfachen Roboter simulieren (Sie kennen diese Simulation vielleicht als „turtle graphics“). Der Roboter bewegt sich auf einer zweidimensionalen Ebene. Sein Zustand besteht aus seiner aktuellen Position und Blickrichtung (im Bogenmaß). Außerdem definieren wir einen Typ von Kommandos, die den Roboterzustand ändern:

```
type State = (Vec, Double) -- (Position, Blickrichtung)
type Command = State -> State
```

(d) Definieren Sie eine Funktion `turn :: Double -> Command`. [1 Punkt]
Der Aufruf `turn dphi state` soll einen Zustand zurückgeben, bei dem sich der Roboter um `dphi` gedreht hat.
Hinweis: Sie müssen nicht dafür sorgen, dass die neue Blickrichtung in $[0, 2\pi]$ liegt.

(e) Definieren Sie eine Funktion `move :: Double -> Command`. [2 Punkte]
Der Aufruf `move dx state` soll einen Zustand zurückgeben, bei dem sich der Roboter um `dx` in seine aktuelle Blickrichtung bewegt hat.
Hinweis: Die neue Position des Roboters ist also gleich der alten Position plus dem Vektor der Länge `dx` in die aktuelle Blickrichtung.

(f) Definieren Sie eine Funktion `perform :: State -> [Command] -> State`. [3 Punkte]
Der Aufruf `perform s0 cs` soll, ausgehend von `s0`, die Kommandos in `cs` anwenden und den finalen Zustand des Roboters zurückgeben.

(g) Definieren Sie eine Funktion `noEffect :: Command -> Bool`. [1 Punkt]
`noEffect` soll genau dann `True` zurückgeben, wenn das übergebene Kommando angewendet auf den Zustand `((0, 0), 0)` den Zustand nicht ändert. Ignorieren Sie Rundungsfehler und vergleichen Sie die Zustände einfach mit ==.

(h) Definieren Sie eine Funktion `optimize :: [Command] -> [Command]`. [4 Punkte]
Der Aufruf `optimize cs` soll aus `cs` Paare von aufeinanderfolgenden Kommandos löschen, die zusammengenommen den Zustand des Roboters gemäß `noEffect` nicht ändern.
Beispiel: `optimize [move 1, move 2, move -2]` ⇒ `[move 1]`

Beispiellösung:

```
vadd :: Vec -> Vec -> Vec
vadd (a, b) (c, d) = (a + c, b + d)

vscale :: Double -> Vec -> Vec
vscale k (x, y) = (k * x, k * y)

unit :: Double -> Vec
unit phi = (cos phi, sin phi)

-- Ab hier waren keine Typannotationen verlangt

move :: Double -> Command
move dx (pos, angle) = (vadd pos (vscale dx (unit angle)), angle)

turn :: Double -> Command
turn dphi (pos, angle) = (pos, angle + dphi)

perform :: State -> [Command] -> State
perform s0 [] = s0
perform s0 (c:cs) = perform (c s0) cs

-- Alternativ
perform' s0 cs = foldr (\f x -> f x) s0 cs
perform'' :: State -> [Command] -> State
perform'' = foldr ($)

noEffect cmd = start == (cmd start)
    where start = ((0, 0), 0)

optimize :: [Command] -> [Command]
optimize (c1:c2:rest) = if noEffect (c2 . c1)
                        then optimize rest
                        else c1 : optimize (c2 : rest)
optimize x = x
```

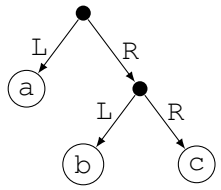
Aufgabe 2 (Haskell: Baumpfade) [13 Punkte]

Gegeben sind folgende Typen für Binärbäume, Richtungen und Pfade:

```
data Tree a = Leaf a | Inner (Tree a) (Tree a)
data Dir = L | R
type Path = [Dir]
```

sowie zwei Beispielbäume

```
t1 = Inner
       (Leaf "a")
       (Inner
         (Leaf "b")
         (Leaf "c"))
```




```
t2 = Leaf "Z"
```

Ein Pfad von der Wurzel eines Baumes zu einem Teilbaum wird durch eine Liste von Richtungen `p :: Path` dargestellt. Jede Pfadrichtung steht dabei für eine ausgehenden Kante; `L` für die linke und `R` für die rechte. Falls `p` zu lang ist, um einen Wurzelpfad in `t` darzustellen, bezeichnen wir `p` als *ungültig*. Zum Beispiel ist `[L]` gültig für `t1`, `[L,R]` hingegen ist ungültig.

(a) Implementieren Sie die Funktion [3 Punkte]

```
focus :: Path -> Tree a -> Tree a
```

sodass der Aufruf `focus p t` den durch den Pfad `p` angegebenen Teilbaum von `t` zurückgibt. Falls `p` ungültig ist, beenden Sie die Ausführung durch Aufruf von `error`.

Beispiel:

```
focus []      t1  ⇒+  Inner (Leaf "a") (Inner (Leaf "b") (Leaf "c"))
focus [R]     t1  ⇒+                   (Inner (Leaf "b") (Leaf "c"))
focus [R,L]   t1  ⇒+                           Leaf "b"
focus [R,L,R] t1  ⇒+  error "invalid path"
```

(b) Implementieren Sie die Funktion [3 Punkte]

```
paths :: Tree a -> [Path]
```

die eine Liste aller gültigen Pfade `p` eines Baumes `t` berechnet.

Beispiel:

```
paths t1 ⇒+ [ [], [L], [R], [R,L], [R,R] ]
paths t2 ⇒+ [ [] ]
```

(c) Implementieren Sie die Funktion [4 Punkte]

```
update :: (Tree a -> Tree a) -> Path -> Tree a -> Tree a
```

Der Aufruf `update f p t` ändert in `t` den durch `p` beschriebenen Teilbaum ab, indem er `f` auf diesen Teilbaum anwendet. Für ungültiges `p` wird `t` unverändert zurückgegeben.

Beispiel:

```
update (focus [L]) [R] t1 ⇒+ Inner (Leaf "a") (Leaf "b")
update (const t2)  [R] t1 ⇒+ Inner (Leaf "a") (Leaf "Z")
```

(d) Verwenden Sie nun `update` zur Implementierung einer Funktion [1 Punkt]

```
set :: Path -> Tree a -> Tree a -> Tree a
```

Der Aufruf `set p t' t` ersetzt in `t` den durch `p` beschriebenen Teilbaum durch `t'`. Für ungültiges `p` wird `t` unverändert zurückgegeben.

Beispiel:

```
set [L]     t2 t1 ⇒+ Inner (Leaf "a") (Leaf "Z")
set [R,L,R] t2 t1 ⇒+ Inner (Leaf "a") (Inner (Leaf "b") (Leaf "c"))
```

(e) Definieren Sie den Baum `neverTooLong :: Tree`, für den *alle* Pfade gültig sind. [2 Punkte]

Beispiellösung:

```
focus :: Path -> Tree a -> Tree a
focus [] t = t
focus (L:p) (Inner l r) = focus p l
focus (R:p) (Inner l r) = focus p r
focus (_:_) (Leaf _)    = error "invalid path"

paths :: Tree a -> [Path]
paths (Leaf _) = [ [] ]
paths (Inner l r) = [] : [ L:p | p <- paths l ] ++ [ R:p | p <- paths r ]

paths2 :: Tree a -> [Path]
paths2 (Leaf _) = [ [] ]
paths2 (Inner l r) = [] : map (L:) (paths l) ++ map (R:) (paths r)

update :: (Tree a -> Tree a) -> Path -> Tree a -> Tree a
update f []    t           = f t
update f (L:p) (Inner l r) = Inner (update f p l) r
update f (R:p) (Inner l r) = Inner l (update f p r)
update f (_:_) t@(Leaf _)  = t

set :: Path -> Tree a -> Tree a -> Tree a
set p t = update (const t) p

neverTooLong :: Tree a
neverTooLong = Inner neverTooLong neverTooLong
```

Aufgabe 3 (Prolog: Rucksack-Problem) [15 Punkte]

Das Rucksack-Problem ist ein bekanntes kombinatorisches Optimierungsproblem. Gegeben ist ein Rucksack mit Maximalgewicht W und eine Liste G von Gegenständen, die jeweils ein Gewicht und einen Wert haben. Eine Teilliste L mit Gegenständen aus G heißt *erlaubte Lösung*, wenn die Summe aller Gewichte der Gegenstände in L nicht größer als W ist. Gesucht ist nun die erlaubte Lösung mit dem größten Wert.

Im Folgenden wird ein einfaches Prolog-Programm entwickelt, das das Rucksack-Problem löst. Hierzu werden zunächst alle erlaubten Lösungen für eine gegebene Liste `G` von Gegenständen und ein Maximalgewicht berechnet sowie anschließend die wertvollste Lösung bestimmt. Wir modellieren einen Gegenstand als Tupel `(Weight, Value)` und nehmen für alle Aufgabenteile an, dass die Gegenstände in `G` verschieden sind.

(a) Definieren Sie ein zweistelliges Prädikat `weight(G, T)`, das das Gesamtgewicht `T` aller Gegenstände in der Liste `G` berechnet. [3 Punkte]

Beispiel:

```
?- weight([(10, 100), (11, 200), (21, 300)], T).
  T = 42.
```

(b) Definieren Sie einen zweistelligen Generator `sublists(G, L)`, der zu einer gegebenen Liste `G` bei Reerfüllung alle Teillisten `L` von `G` generiert. `L` ist Teilliste von `G`, wenn sich `L` durch Entfernen von Einträgen aus `G` erhalten lässt. [4 Punkte]

Beispiel:

```
?- sublists([1, 2], L).
  L = [1, 2] ; L = [1] ; L = [2] ; L = [] ; No
```

(c) Definieren Sie einen dreistelligen Generator `rucksack(G, MaxW, S)`, welcher für eine Liste `G` von Gegenständen und ein Maximalgewicht `MaxW` bei Reerfüllung alle erlaubten Lösungen `S` von `G` generiert. [3 Punkte]
Hinweis: Die Prädikate aus den vorherigen Aufgabenteilen können nützlich sein.

(d) Definieren Sie den Generator `better(L, R, B)`, der für zwei Lösungen `L` und `R` die Lösung `B` mit dem größten Gesamtwert berechnet. Falls beide Lösungen den gleichen Gesamtwert haben, soll `B` mit beiden Lösungen reerfüllt werden. Sie dürfen dabei das Prädikat `value(G, V)` als gegeben annehmen, das den Gesamtwert `V` aller Gegenstände in der Liste `G` berechnet. [3 Punkte]

(e) Sei `A` eine nichtleere Liste von erlaubten Lösungen. Definieren Sie ein zweistelliges Prädikat `best(A, S)`, das die wertvollsten erlaubten Lösungen `S` aus `A` bestimmt. [2 Punkte]
Hinweis: Das Prädikat `better(L, R, B)` könnte nützlich sein.

Beispiellösung:

```
weight([], 0).
weight([(W, _)|R], TotalWeight) :- weight(R, WR), TotalWeight is WR + W.

sublists([], []).
sublists([X|L], [X|L2]) :- sublists(L, L2).
sublists([_|L], L2) :- sublists(L, L2).

% Alternativlösung mit delete aus der Vorlesung
sublists2(G, G).
sublists2(G, L) :- delete(G, _, G2), sublists2(G2, L).

rucksack(Items, MaxWeight, Taken) :- sublists(Items, Taken),
                                     weight(Taken, Weight),
                                     Weight =< MaxWeight.

better(X, Y, X) :- value(X, VX), value(Y, VY), VX >= VY.
better(X, Y, Y) :- value(X, VX), value(Y, VY), VX =< VY.

best([X], X).
best([L|R], S) :- best(R, SR), better(L, SR, S).

% Alternativlösung durch Listenänderung
best2([X], X).
best2([X,Y|Z], S) :- value(X, VX), value(Y, VY), VX >= VY,
                             best2([X|Z], S).
best2([X,Y|Z], S) :- value(X, VX), value(Y, VY), VX =< VY,
                             best2([Y|Z], S).
```

Aufgabe 4 (Lambdakalkül, Maybe-Datentyp) [14 Punkte]

Aus der Vorlesung kennen Sie die Modellierung von Church-Zahlen und Church-Booleans. Auch der `Maybe`-Datentyp aus Haskell lässt sich im Lambdakalkül darstellen. In Haskell ist dieser wie folgt definiert:

```
data Maybe a = Nothing | Just a
```

Im λ-Kalkül können die Konstruktoren definiert werden durch:

$$\begin{aligned} \texttt{nothing} &= \lambda \texttt{n}.\ \lambda \texttt{j}.\ \texttt{n} \\ \texttt{just} &= \lambda \texttt{x}.\ \lambda \texttt{n}.\ \lambda \texttt{j}.\ \texttt{j}\ \texttt{x} \end{aligned}$$

Außerdem sei der Term `maybe` definiert als:

$$\texttt{maybe} = \lambda \texttt{n}.\ \lambda \texttt{j}.\ \lambda \texttt{m}.\ \texttt{m}\ \texttt{n}\ \texttt{j}$$

(a) Zeigen Sie, dass für beliebige `n`, `j` und `x` gilt: [6 Punkte]

$$\begin{aligned} & \texttt{maybe}\ n\ j\ \texttt{nothing} \Rightarrow^* n \\ \text{und}\quad & \texttt{maybe}\ n\ j\ (\texttt{just}\ x) \Rightarrow^* j\ x \end{aligned}$$

Fassen Sie dabei *keine* Schritte der β-Reduktion zusammen.

(b) Definieren Sie eine λ-Funktion `isNothing` mit folgenden Eigenschaften: [2 Punkte]

$$\begin{aligned} \texttt{isNothing nothing} &\Rightarrow^* c_{\mathsf{true}} \\ \texttt{isNothing (just x)} &\Rightarrow^* c_{\mathsf{false}} \end{aligned}$$

(c) Definieren Sie eine $\lambda-$Funktion `maybeMap`, welche eine Funktion auf den `Maybe`-Typen erweitert, d.h. es soll gelten [2 Punkte]

$$\begin{aligned} & \texttt{maybeMap f nothing} \Rightarrow^* \texttt{nothing} \\ \text{und}\quad & \texttt{maybeMap f (just y)} \Rightarrow^* \texttt{just (f y)}. \end{aligned}$$

(d) Definieren Sie eine $\lambda-$Funktion `orElse`, welche folgenden Reduktionseigenschaften besitzt: [4 Punkte]

$$\begin{aligned} \texttt{orElse (just x) y} &\Rightarrow^* \texttt{just x}, \\ \texttt{orElse nothing (just y)} &\Rightarrow^* \texttt{just y} \text{ und} \\ \texttt{orElse nothing nothing} &\Rightarrow^* \texttt{nothing}. \end{aligned}$$

Beispiellösung:

(a)

$$
\begin{aligned}
\texttt{maybe n j nothing} &= (\lambda\texttt{n}.\ \lambda\texttt{j}.\ \lambda\texttt{m}.\ \texttt{m n j})\ n\ j\ \texttt{nothing} \\
&\Rightarrow (\lambda\texttt{j}.\ \lambda\texttt{m}.\ \texttt{m n j})\ j\ \texttt{nothing} \\
&\Rightarrow (\lambda\texttt{m}.\ \texttt{m n j})\ \texttt{nothing} \\
&\Rightarrow \texttt{nothing}\ n\ j \\
&= (\lambda\texttt{n}.\ \lambda\texttt{j}.\ \texttt{n})\ n\ j \\
&\Rightarrow (\lambda\texttt{j.}\ \texttt{n})\ \texttt{j} \\
&\Rightarrow n \\
\texttt{maybe}\ n\ j\ (\texttt{just}\ x) &= (\lambda\texttt{n}.\ \lambda\texttt{j}.\ \lambda\texttt{m}.\ \texttt{m n j})\ n\ j\ (\texttt{just}\ x) \\
&\Rightarrow (\lambda\texttt{j.}\ \lambda\texttt{m.}\ \texttt{m n j})\ \texttt{j}\ (\texttt{just x}) \\
&\Rightarrow (\lambda\texttt{m.}\ \texttt{m n j})\ (\texttt{just x}) \\
&\Rightarrow \texttt{just}\ x\ n\ j \\
&= (\lambda\texttt{x}.\ \lambda\texttt{n}.\ \lambda\texttt{j}.\ \texttt{j x})\ x\ n\ j \\
&\Rightarrow (\lambda\texttt{n.}\ \lambda\texttt{j.}\ \texttt{j x})\ \texttt{n j} \\
&\Rightarrow (\lambda\texttt{j.}\ \texttt{j x})\ \texttt{j} \\
&\Rightarrow j\ x
\end{aligned}
$$

(b) `isNothing = maybe true (λy. false)`

(c) `maybeMap = λf. maybe nothing (λy. just (f y))`

(d) `orElse = λa. λb. maybe b just a`

Aufgabe 5 (Typinferenz) [17 Punkte]

(a) Geben Sie den allgemeinsten Typen des Terms `λf. λx. f x x` an. [2 Punkte]

(b) Stellen Sie den Typherleitungsbaum für den Term `λf. λx. f (x x)` auf. Das Constraintsytem brauchen Sie nicht aufzustellen. [9 Punkte]

(c) Zeigen Sie mit dem Typinferenzalgorithmus, dass `λf. λx. f (x x)` nicht typisierbar ist. Verwenden Sie hierzu Constraints, die sich aus dem aufgestellten Typherleitungsbaum ergeben. Geben Sie die konkrete Bedingung an, wegen der die Typinferenz fehlschlägt. [4 Punkte]
Hinweis: Sie müssen nicht das komplette Constraintsystem aufstellen und so weit wie möglich unifizieren. Es genügen die Constraints und Unifikationsschritte, die zum Fehler führen.

(d) Geben Sie einen λ-Term f^* an, sodass folgende Bedingungen gelten: [2 Punkte]
- Der Ausdruck
$$(\lambda \mathtt{f.}\ \lambda \mathtt{x.}\ \mathtt{f}\ (\mathtt{x}\ \mathtt{x}))\ f^*$$
reduziert mit voller β-Reduktion zu einem Term t.
- t ist geschlossen (d. h. enthält keine freien Variablen).
- t ist typisierbar.

Beispiellösung:

(a) $\lambda \mathtt{f.}\ \lambda \mathtt{x.}\ \mathtt{f}\ \mathtt{x}\ \mathtt{x} : (\alpha \to \alpha \to \beta) \to \alpha \to \beta$

(b)

$$\dfrac{\dfrac{\dfrac{\dfrac{\Gamma(\mathtt{f}) = \alpha_2}{\Gamma \vdash \mathtt{f} : \alpha_6}\,\mathit{Var} \qquad \dfrac{\dfrac{\Gamma(\mathtt{x}) = \alpha_4}{\Gamma \vdash \mathtt{x} : \alpha_8}\,\mathit{Var} \qquad \dfrac{\Gamma(\mathtt{x}) = \alpha_4}{\Gamma \vdash \mathtt{x} : \alpha_9}\,\mathit{Var}}{\Gamma \vdash \mathtt{x}\ \mathtt{x} : \alpha_7}\,\mathit{App}}{\mathtt{f} : \alpha_2,\ \mathtt{x} : \alpha_4 =: \Gamma \vdash \mathtt{f}\ (\mathtt{x}\ \mathtt{x}) : \alpha_5}\,\mathit{App}}{\mathtt{f} : \alpha_2 \vdash \lambda \mathtt{x.}\ \mathtt{f}\ (\mathtt{x}\ \mathtt{x}) : \alpha_3}\,\mathit{Abs}}{\vdash \lambda \mathtt{f.}\ \lambda \mathtt{x.}\ \mathtt{f}\ (\mathtt{x}\ \mathtt{x}) : \alpha_1}\,\mathit{Abs}$$

(c) Aus dem Herleitungsbaum erhalten wir (unter anderem):

$$\alpha_4 = \alpha_8$$
$$\alpha_4 = \alpha_9$$
$$\alpha_8 = \alpha_9 \to \alpha_7$$

Nach Einsetzen der ersten beiden Constraints in den dritten ergibt sich:

$$\alpha_4 = \alpha_4 \to \alpha_7$$

Dieser Constraint ist nicht unifizierbar, weil die Bedingung $\alpha_4 \notin FV(\alpha_4 \to \alpha_7)$ („occurs check“) nicht erfüllt ist.

(d) $f^* = \lambda \mathtt{y.}\ (\lambda \mathtt{z.}\ \mathtt{z})$ (dann ist $t = \lambda \mathtt{x.}\ \lambda \mathtt{z.}\ \mathtt{z}$)

Aufgabe 6 (MPI) [14 Punkte]

Gegeben sei die untenstehende Funktion `calculate`, die als Parameter die zwei Arrays `pointA` und `pointB` sowie mit `x` die Anzahl der Elemente in ihnen entgegennimmt. Gehen Sie davon aus, dass MPI korrekt initialisiert und finalisiert wird. Gehen Sie weiter davon aus, dass `x` ein Vielfaches der Anzahl Prozesse ist und dass die beiden Arrays `pointA` und `pointB` korrekt initialisiert wurden sowie überlappungsfrei im Speicher liegen. Die verwendete Funktion `sqrt(double arg)` nimmt als Argument einen Wert vom Typ `double` entgegen und gibt die Wurzel des Werts als `double` zurück.

```
1  double calculate(int x, double pointA[], double pointB[]) {
2      int rank, size;
3      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4      MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6      double result = 0.0;
7
8      MPI_Bcast(pointA, x, MPI_DOUBLE, 0, MPI_COMM_WORLD);
9      MPI_Bcast(pointB, x, MPI_DOUBLE, 0, MPI_COMM_WORLD);
10
11     int elements_per_process = x / size;
12     int start_dim = rank * elements_per_process;
13     int end_dim = start_dim + elements_per_process;
14     double local_result = 0.0;
15     for (int i = start_dim; i < end_dim; i++) {
16         double diff = pointA[i] - pointB[i];
17         local_result += diff * diff;
18     }
19     MPI_Reduce(&local_distance, &distance, 1, MPI_DOUBLE, MPI_SUM, 0,
          MPI_COMM_WORLD);
20     result = sqrt(result);
21     return result;
22 }
```

(a) Füllen Sie in den nachfolgenden Tabellen aus, wie die jeweils angegebenen Variablen nach Ausführung der in den Tabellen gegebenen Quelltext-Zeile belegt sind. Nehmen Sie an, dass die Arrays [6 Punkte]
`pointA[] = {1.0, 2.0, 3.0, 0.0, 2.0, 1.0, 7.0, 0.0}`
`pointB[] = {2.0, 5.0, 5.0, 5.0, 6.0, 2.0, 10.0, 4.0}`
mit `x = 8` übergeben werden und dass das Programm mit 4 Prozessen bearbeitet wird.

Beispiellösung: Siehe Tabellen.

(a) **Variable: `pointA`**

Prozess Nr.	**Wert nach Zeile 8 (Lösung)**
0	1.0, 2.0, 3.0, 0.0, 2.0, 1.0, 7.0, 0.0
1	1.0, 2.0, 3.0, 0.0, 2.0, 1.0, 7.0, 0.0
2	1.0, 2.0, 3.0, 0.0, 2.0, 1.0, 7.0, 0.0
3	1.0, 2.0, 3.0, 0.0, 2.0, 1.0, 7.0, 0.0

(b) **Variable: `pointB`**

Prozess Nr.	**`Wert nach Zeile 9` (Lösung)**
0	2.0, 5.0, 5.0, 5.0, 6.0, 2.0, 10.0, 4.0
1	2.0, 5.0, 5.0, 5.0, 6.0, 2.0, 10.0, 4.0
2	2.0, 5.0, 5.0, 5.0, 6.0, 2.0, 10.0, 4.0
3	2.0, 5.0, 5.0, 5.0, 6.0, 2.0, 10.0, 4.0

(c) **Variable: `start_dim`**

Prozess Nr.	**`Wert nach Zeile 13` (Lösung)**
0	0
1	2
2	4
3	6

(d) **Variable: `end_dim`**

Prozess Nr.	**`Wert nach Zeile 13` (Lösung)**
0	2
1	4
2	6
3	8

(e) **Variable: `local_result`**

Prozess Nr.	**`Wert nach Zeile 18` (Lösung)**
0	10
1	29
2	17
3	25

(f) **Variable: `result`**

Prozess Nr.	**Wert nach Zeile 20 (Lösung)**
0	9
1	0
2	0
3	0

(b) Beschreiben Sie, welche Funktionalität von `calculate` im Allgemeinen realisiert wird. Reine Erläuterungen der Programmabfolge im konkreten Beispielfall sind keine gültige Lösung und werden nicht bepunktet. [1 Punkt]

Beispiellösung: Euklidische Distanz; Wurzel der Summe der paarweise quadrierten Differenzen; Formel für Berechnung der euklidischen Distanz ist auch ausreichend.

(c) Ist die Annahme, dass die Anzahl Elemente der Arrays ein Vielfaches der Anzahl Prozesse ist, notwendig für die oben gezeigte Implementierung der Funktion `calculate`? Begründen Sie Ihre Antwort kurz. [2 Punkte]

Beispiellösung: Ja, da `elements_per_process` als Integer gespeichert wird. Daher muss das Ergebnis von `x / size` ganzzahlig sein, sonst kommt es zu Fehlern, da überschüssige Elemente nicht verteilt werden.

(d) Sind die MPI_Bcast-Aufrufe in Zeile 8 und 9 notwendig? Begründen Sie. [2 Punkte]

Beispiellösung: Nur, wenn pointA und pointB nur auf Prozess 0 vorliegen. Da im Allgemeinen nicht davon ausgegangen werden kann, dass Variablen auf allen Prozessen initialisiert sind, also ja. Alternativ: Nein, pointA und pointB liegen auf allen Prozessen bereits vor.

(e) Was ändert sich am Wert der `result`-Variable nach Zeile 20, wenn der Aufruf zu MPI_Reduce durch einen Aufruf zu MPI_Allreduce ersetzt wird? [1 Punkt]

Beispiellösung: Das Ergebnis wird auf allen Prozessen vorliegen.

(f) Was ist jeweils ein Vor- und Nachteil von blockierender Kommunikation? [2 Punkte]

Beispiellösung: Vorteil: Nach dem Aufruf ist direkt klar, dass Ergebnisse vorliegen; daher auch einfacher zu programmieren, da alle Puffer verwendet werden können.
Nachteil: uU schlechtere Performance, Deadlock-Gefahr
Kein Vorteil ist synchrone Kommunikation, da synchrone Kommunikation != blockierender Kommunikation.

Aufgabe 7 (Akka-Aktoren, Java, Design by Contract) [11 Punkte]

Gegeben sind die beiden untenstehenden Klassen, die ein Ticketbuchungssystem implementieren. Die Klasse `TicketBookingSystem` sendet Anfragen zum Ticketkauf an den `TicketSellerActor`, der die vorhandenen Tickets verwaltet. Das Ergebnis des Buchungsvorgangs wird vom `CustomerActor` ausgegeben. Es sollen insgesamt im gesamten System maximal `availableTickets = 10` verkauft werden.

```
1  public class TicketBookingSystem {
2      public static void main(String[] args) {
3          ActorSystem system = ActorSystem.create("TicketBookingSystem");
4          ActorRef ticketSellerActor = system.actorOf(Props.create(
5                                  TicketSellerActor.class),
6                                  "ticketSellerActor");
7          ActorRef customerActor = system.actorOf(Props.create(
8                                  CustomerActor.class),
9                                  "customerActor");
10         ticketSellerActor.tell(2, customerActor);
11 } }
12
13 public class CustomerActor extends AbstractActor {
14     public Receive createReceive() {
15         return receiveBuilder()
16         .match(String.class, message -> {
17             System.out.println("Booking outcome: " + message);
18         }).build();
19 } }
20
21 public class TicketSellerActor extends AbstractActor {
22     private int availableTickets = 10;
23
24     public Receive createReceive() {
25         return receiveBuilder().match(                    -> {
26             if (availableTickets >= numTickets) {
27
28             } else {
29
30             }
31         }).build();
32 } }
```

(a) Füllen Sie die Lücken in der `createReceive`-Methode des `TicketSellerActors` aus. Sie können davon ausgehen, dass bei diesem Aktor eingehende Nachrichten immer vom Typ `Integer` sind. Die Methode soll eine Nachricht an den `CustomerActor` senden, welcher der Sender der Nachricht ist. Diese Nachricht soll einen Text beinhalten, der angibt, ob die Ticketbuchung erfolgreich war. Falls sie erfolgreich war, soll außerdem die Zahl der verfügbaren Tickets entsprechend aktualisiert werden. [3.5 Punkte]

Beispiellösung:

```
return receiveBuilder().match(Integer.class, numTickets -> {
    if (availableTickets >= numTickets) {
        availableTickets -= numTickets;
        //NULL can be used instead of getSelf()
        getSender().tell("Tickets booked successfully.", getSelf());
    } else {
        //NULL can be used instead of getSelf()
        getSender().tell("Not enough tickets available.", getSelf());
    }
}).build();
```

(b) Um mehr Skalierbarkeit sicherzustellen, werden nun zwei `TicketSellerActors` erstellt. Gehen Sie davon aus, dass die Ausgabe für eine erfolgreiche Ticketbuchung „Booked X tickets successfully“ ist, für eine nicht erfolgreiche Buchung „Could not book X tickets“. Was ist die Ausgabe des Systems für untenstehendes Szenario? Begründen Sie, warum es zu dieser Ausgabe kommt.
Entspricht das der oben vorgesehenen Spezifikation, dass im gesamten Aktor-System maximal 10 Tickets verkauft werden sollen? [3 Punkte]

```
ticketSellerActor.tell(2, customerActor);
ticketSellerActor2.tell(7, customerActor);
ticketSellerActor.tell(3, customerActor);
```

Beispiellösung:

```
Booked 2 tickets successfully
Booked 7 tickets successfully
Booked 3 tickets successfully
```

Es kommt bei allen zur Ausgabe, dass die Ticketbuchung erfolgreich war, da jeder der `TicketSellerActors` seinen eigenen Speicher hat und daher auch jeweils 10 Tickets vorrätig. Dies entspricht nicht der Spezifikation, dass maximal 10 Tickets verkauft werden sollen.

(c) Schreiben Sie eine Vor- und eine Nachbedingung in JML für die nachfolgend gegebene Implementierung der `sellTickets`-Methode. Die trivialen Vor- und Nachbedingungen `true` und `false` sind keine gültigen Lösungen und werden nicht bepunktet. [2 Punkte]

```
public class TicketSeller {
    int availableTickets;
    public void sellTicket(int numTickets) {
        availableTickets = availableTickets - numTickets;
    }
}
```

Beispiellösung: Vorbedingung: `requires availableTickets >= 0;` Nachbedingung: `ensures availableTickets = old(availableTickets) - numTickets;`

(d) Kann die in Teilaufgabe `(c)` gegebene Methode `sellTickets` sicher durch zwei Threads gleichzeitig aufgerufen werden? Begründen Sie kurz. Falls nicht, erklären Sie, wie das sichere Aufrufen durch mehrere Threads gleichzeitig ermöglicht werden könnte. [2.5 Punkte]

Beispiellösung: Nein, da beim Zugriff auf `availableTickets` eine Race Condition auftritt, da die Subtraktion nicht atomar ist. Der kritische Abschnitt muss geschützt werden - zB Methode als `synchronized` definieren.

Aufgabe 8 (SLL-Grammatiken) [6 Punkte]

In Haskell gibt es acht Arten von Ausdrücken, die mit `[` beginnen:

- `[ ]`
- `[` E `]`
- `[` E `,` E `]` (längere Listen ignorieren wir)
- `[` E `..` `]`
- `[` E `,` E `..` `]`
- `[` E `..` E `]`
- `[` E `,` E `..` E `]`
- `[` E `|` E `]`

Die hier relevanten Terminale sind `[`, `]`, `|`, `..` und `,`. Markieren Sie in Ihrer Lösung Terminale, z. B. durch Unterstreichen.

E ist das Nichtterminal für Ausdrücke (Expressions). Sie dürfen annehmen, dass `]` $\notin \text{First}(E)$.

Es sei L das Nichtterminal für alle Ausdrücke, die mit `[` beginnen. Geben Sie eine Grammatik für L in SLL(1)-Form an.

Beispiellösung:

$$
\begin{aligned}
L &\to \texttt{[}\ L_1 \\
L_1 &\to \texttt{]} \mid E\ L_2 \\
L_2 &\to \texttt{]} \mid \texttt{,}\ E\ L_3 \mid \texttt{..}\ L_4 \mid \texttt{|}\ E\ \texttt{]} \\
L_3 &\to \texttt{]} \mid \texttt{..}\ L_5 \\
L_4 &\to \texttt{]} \mid E\ \texttt{]} \\
L_5 &\to \texttt{]} \mid E\ \texttt{]}
\end{aligned}
$$

Aufgabe 9 (Java-Bytecode) [13 Punkte]

Gegeben sei folgender Java-Bytecode einer Methode `func`. Rekonstruieren Sie eine gültige Java-Methodendefinition, für die dieser Bytecode erzeugt wird.

Hinweise:

- Vergessen Sie nicht den Methodenkopf.
- Es bieten sich folgende Variablennamen an:

1. Argument:	`n`
2. Argument:	`k`
lokale Variable Nr. 3:	`acc`
lokale Variable Nr. 4:	`i`

- Die Bytecode-Instruktion `iinc` v, d addiert d auf den Wert der lokalen Variable Nr. v, ohne den Stack zu verändern.

```
func:(II)I {
     0: iload_2
     1: iconst_1
     2: if_icmple     10
     5: iload_2
     6: iload_1
     7: if_icmplt     12
    10: iconst_1
    11: ireturn
    12: iconst_1
    13: istore_3
    14: iconst_1
    15: istore        4
    17: iload         4
    19: iload_2
    20: if_icmpgt     39
    23: iload_3
    24: iload_1
    25: imul
    26: iload         4
    28: idiv
    29: istore_3
    30: iinc          4, 1
    33: iinc          1, -1
    36: goto          17
    39: iload_3
    40: ireturn
}
```

Beispiellösung: **Beispiellösung:**

```
1  public class Aufgabe {
2      // func berechnet den Binomialkoeffizienten "n über k"
3      int func(int n, int k) {
4          if (k <= 1 || k >= n) {
5              return 1;
6          }
7
8          int acc = 1;
9          int i = 1;
10
11         while (i <= k) {
12             acc = acc * n / i;
13             i++;
14             n--;
15         }
16
17         return acc;
18     }
19 }
```